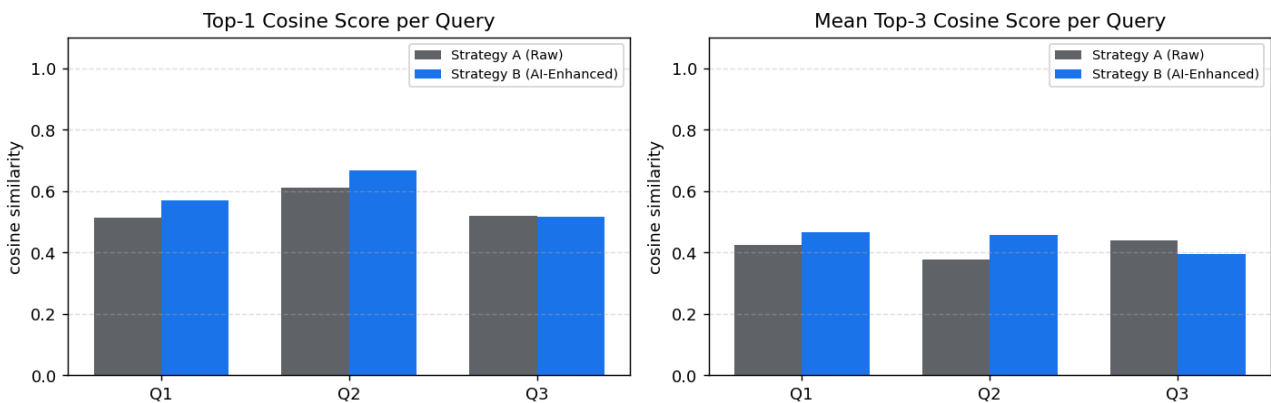


Retrieval Benchmark - Strategy A vs Strategy B

Auto-generated comparison of Raw Vector Search (A) vs AI-Enhanced Retrieval (B).

- Top-K: 3
- Similarity metric: **cosine**
- Embedding model: **sentence-transformers/all-MiniLM-L6-v2** (mocks Vertex textembedding-gecko)
- Vector store: **FAISS** IndexFlatIP (file-backed, PDF-compliant)
- Query expander: **anthropic** (claude-haiku-4-5-20251001)
- Queries evaluated: 3

Strategy A vs Strategy B: cosine similarity comparison



Aggregate Results

Metric	Strategy A (Raw)	Strategy B (AI-Enhanced)	Delta
Mean top-1 cosine score	0.5480	0.5845	+0.0365
Mean top-3 cosine score	0.4132	0.4390	+0.0258
Hit Rate @ 3	1.0000	1.0000	+0.0000
MRR	1.0000	1.0000	+0.0000
nDCG @ 3	1.0000	0.9218	-0.0782
Mean latency (ms)	41.2	3475.9	+3434.6

Per-query winner: Strategy A wins 1, Strategy B wins 2, ties 0 (threshold: cosine delta > 0.01)

Per-Query Summary

#	Query	Winner	Mean A	Mean B	Uplift	Hit@3 A/B	MRR A/B
Q1	How does the system handle peak load?	B	0.4238	0.4661	+0.0422	1.00/1.00	1.00/1.00
Q2	What happens when too many users hit the API at on...	B	0.3773	0.4574	+0.0801	1.00/1.00	1.00/1.00
Q3	Explain the failover mechanism during outages.	A	0.4384	0.3936	-0.0448	1.00/1.00	1.00/1.00

Query 1: "How does the system handle peak load?"

Strategy B expanded query: How does the system manage handle process peak load traffic surge high demand maximum capacity stress testing performance under load scalability load balancing resource allocation during peak hours traffic spikes concurrent users server capacity bottleneck mitigation auto-scaling elastic scaling throughput latency response time under heavy load infrastructure scaling cloud resources capacity planning performance optimization during peak traffic periods

Rank	Strategy A (Raw)	Score	Strategy B (Expanded)	Score
1	The system handles peak load through horizontal auto-scaling powered by Kubernetes...	0.5135	The system handles peak load through horizontal auto-scaling powered by Kubernetes...	0.5693
2	Incoming traffic is distributed across regional load balancers operating at both...	0.3939	APIs are protected by a tiered rate-limiting system using token-bucket and leaky...	0.4178
3	APIs are protected by a tiered rate-limiting system using token-bucket and leaky...	0.3641	Incoming traffic is distributed across regional load balancers operating at both...	0.4111

Latency A: embed 77.5ms / search 0.2ms - _Latency B: expand 3125.0ms / embed 64.8ms / search 0.3ms_

Query 2: "What happens when too many users hit the API at once?"

Strategy B expanded query: What happens when too many concurrent users simultaneously access the API at the same time? How does the system handle high traffic load, traffic spikes, and request surges? What is the impact of exceeding rate limits, throttling, and API capacity? How do rate limiting, request queuing, and traffic management work? What are the consequences of overload conditions, server saturation, and resource exhaustion? How does the system respond to DDoS attacks, traffic congestion, and bottlenecks? What happens during peak usage, burst traffic, and sudden demand? How are requests

Rank	Strategy A (Raw)	Score	Strategy B (Expanded)	Score
1	APIs are protected by a tiered rate-limiting system using token-bucket and leaky...	0.6124	APIs are protected by a tiered rate-limiting system using token-bucket and leaky...	0.6665
2	Incoming traffic is distributed across regional load balancers operating at both...	0.2687	Incoming traffic is distributed across regional load balancers operating at both...	0.3564
3	The system handles peak load through horizontal auto-scaling powered by Kubernetes...	0.2509	The system handles peak load through horizontal auto-scaling powered by Kubernetes...	0.3493

Latency A: embed 28.2ms / search 0.1ms - _Latency B: expand 4178.5ms / embed 63.6ms / search 0.1ms_

Query 3: "Explain the failover mechanism during outages."

Strategy B expanded query: Explain the failover mechanism during outages, system failures, and service disruptions. How does automatic failover work? What are failover strategies, redundancy mechanisms, and backup systems? Discuss failover protocols, high availability architectures, disaster recovery procedures, system resilience, and fault tolerance techniques. Include information about failover detection, failover triggers, switchover processes, recovery procedures, downtime mitigation, business continuity planning, and infrastructure failover handling during network outages, server failures, database failures, and application crashes.

Rank	Strategy A (Raw)	Score	Strategy B (Expanded)	Score
1	The primary database operates in a multi-AZ configuration with synchronous repli...	0.5181	The primary database operates in a multi-AZ configuration with synchronous repli...	0.5178

Rank	Strategy A (Raw)	Score	Strategy B (Expanded)	Score
2	Disaster recovery procedures define an RPO of 5 minutes and an RTO of 30 minutes...	0.4372	Disaster recovery procedures define an RPO of 5 minutes and an RTO of 30 minutes...	0.4152
3	Incoming traffic is distributed across regional load balancers operating at both...	0.3599	The system handles peak load through horizontal auto-scaling powered by Kubernetes...	0.2478

Latency A: embed 17.6ms / search 0.1ms - _Latency B: expand 2927.6ms / embed 67.5ms / search 0.1ms_

Methodology

- **Hit Rate @ K** - fraction of queries where at least one of the top-K retrieved documents is in the ground-truth relevant set.
- **MRR (Mean Reciprocal Rank)** - 1 / position of the first relevant result, averaged across queries.
- **nDCG @ K** - normalized discounted cumulative gain, rewards relevant results appearing at higher ranks.
- **Ground truth** - hand-labeled per query in `app/Services/benchmark_metrics.py :: GROUND_TRUTH`.
- **Winner** - strategy with higher mean cosine score across the top-K results (threshold 0.01 to flag ties).
- **Latency** - end-to-end retrieval time excluding generation. Strategy B includes an additional LLM call for query expansion.

Raw JSON

```
{
  "top_k": 3,
  "queries": [
    {
      "query": "How does the system handle peak load?",
      "strategy_a": {
        "results": [
          {
            "chunk_id": "a7842a68-0-50471deeed4cebd2",
            "doc_id": "a7842a680dad",
            "text": "The system handles peak load through horizontal auto-scaling powered by Kubernetes",
            "source": "auto_scaling.txt",
            "score": 0.5135,
            "rank": 1,
            "strategy": "A"
          },
          {
            "chunk_id": "2fe17f67-0-cd5d5655f1a85fa7",
            "doc_id": "2fe17f670bf6",
            "text": "Incoming traffic is distributed across regional load balancers operating at both L"
```

```

        "source": "load_balancer.txt",
        "score": 0.3939,
        "rank": 2,
        "strategy": "A"
    },
    {
        "chunk_id": "f70f2662-0-0d820cc207d156bf",
        "doc_id": "f70f266260ee",
        "text": "APIs are protected by a tiered rate-limiting system using token-bucket and leaky-b",
        "source": "rate_limiting.txt",
        "score": 0.3641,
        "rank": 3,
        "strategy": "A"
    }
],
"timings_ms": {
    "embed_ms": 77.45780004188418,
    "search_ms": 0.19569997675716877,
    "expand_ms": 0.0
}
},
"strategy_b": {
    "expanded_query": "How does the system manage handle process peak load traffic surge high demand",
    "results": [
        {
            "chunk_id": "a7842a68-0-50471deeed4cebd2",
            "doc_id": "a7842a680dad",
            "text": "The system handles peak load through horizontal auto-scaling powered by Kubernetes",
            "source": "auto_scaling.txt",
            "score": 0.5693,
            "rank": 1,
            "strategy": "B"
        },
        {
            "chunk_id": "f70f2662-0-0d820cc207d156bf",
            "doc_id": "f70f266260ee",
            "text": "APIs are protected by a tiered rate-limiting system using token-bucket and leaky-b",
            "source": "rate_limiting.txt",
            "score": 0.4178,
            "rank": 2,
            "strategy": "B"
        },
        {
            "chunk_id": "2fe17f67-0-cd5d5655f1a85fa7",
            "doc_id": "2fe17f670bf6",
            "text": "Incoming traffic is distributed across regional load balancers operating at both L",
            "source": "load_balancer.txt",
            "score": 0.4111,
            "rank": 3,
            "strategy": "B"
        }
    ],
    "timings_ms": {
        "embed_ms": 64.82929992489517,
        "search_ms": 0.25349995121359825,
        "expand_ms": 3125.04870002158
    }
}
},
{
    "query": "What happens when too many users hit the API at once?",
    "strategy_a": {
        "results": [
            {
                "chunk_id": "f70f2662-0-0d820cc207d156bf",

```

```

    "doc_id": "f70f266260ee",
    "text": "APIs are protected by a tiered rate-limiting system using token-bucket and leaky-b
    "source": "rate_limiting.txt",
    "score": 0.6124,
    "rank": 1,
    "strategy": "A"
  },
  {
    "chunk_id": "2fe17f67-0-cd5d5655f1a85fa7",
    "doc_id": "2fe17f670bf6",
    "text": "Incoming traffic is distributed across regional load balancers operating at both L
    "source": "load_balancer.txt",
    "score": 0.2687,
    "rank": 2,
    "strategy": "A"
  },
  {
    "chunk_id": "a7842a68-0-50471deeed4cebd2",
    "doc_id": "a7842a680dad",
    "text": "The system handles peak load through horizontal auto-scaling powered by Kubernetes
    "source": "auto_scaling.txt",
    "score": 0.2509,
    "rank": 3,
    "strategy": "A"
  }
],
"timings_ms": {
  "embed_ms": 28.187799965962768,
  "search_ms": 0.11939997784793377,
  "expand_ms": 0.0
}
},
"strategy_b": {
  "expanded_query": "What happens when too many concurrent users simultaneously access the API at
  "results": [
... [truncated - see retrieval_benchmark.json for full JSON]

```